PROJECT TECHNICAL REPORT

Smart-Parking Edge Box Simulation

Zero-Trust Cryptographic Infrastructure & Real-World Edge Architecture

An Autonomous Edge Computing System for Privacy-Preserving Smart Cities

Domain:	Edge Computing, Applied Cryptography & IoT Security
Scenario:	Municipal Smart Parking Facility with Local Edge Box
Repository:	https://github.com/swethasweth19/SmartPark-Edge-Simulator
Implementation:	Python 3.11, PyCryptodome, Paillier (phe), Flask
Cryptography:	AES-256-GCM, IBE, ABE, PRE, FE, Honey Encryption, Searchable Encryption (SSE), Paillier HE, JWT
Simulation UI:	Interactive 2D Real-World Parking Lot with Gates & Sensors
Date:	October 2026
Status:	Fully Implemented, Verified & Deployed

EXECUTIVE SUMMARY

This project models and implements an intelligent on-site Smart-Parking Edge Box that processes camera and ultrasonic sensor telemetry locally. It addresses the vulnerabilities of traditional cloud architectures: driver location surveillance, sensor spoofing, payment terminal brute-forcing, and camera flood attacks. By combining 11 advanced cryptographic algorithms with an interactive real-world 8-bay parking lot simulator, the system delivers zero-trust privacy and active cyber defense on edge hardware.

1. PROBLEM STATEMENT & SYSTEM ARCHITECTURE

In smart cities, roadside Automated License Plate Readers (ALPR) and ultrasonic occupancy sensors continuously record vehicle movements. Transmitting this raw telemetry to distant cloud servers exposes citizens to mass surveillance and tracking. Furthermore, outdoor sensor lines are vulnerable to packet injection, payment kiosks face PIN brute-force attacks, and compromised cameras can flood the local network with malicious video frames to crash barrier gates.

Component Subsystems & Source Files

Subsystem	File	Cryptographic & Functional Responsibilities
Gate Camera	`camera.py`	Captures plates, blurs & hashes, signs with ECDSA P-256, encrypts with AES-
Bay Sensor	`sensor.py`	Ultrasonic sensor process. Tracks bays, reduces GPS coords, rotates IDs, sign
Edge Server	`parking_edge.py`	The on-site edge security box. Ingests packets, manages ABE, PRE, FE, SSE,
Crypto Engine	`crypto_utils.py`	Modular engine for all 11 crypto primitives (AES-GCM, IBE, ABE, PRE, FE, Ho
Simulator UI	`app.py / index.html`	Interactive visual parking lot dashboard with boom barriers, cars, sensors & pa

2. THE 11 CRYPTOGRAPHIC SPECIFICATIONS (OVERVIEW)

Step 1: AES-256-GCM + ECDSA	Per-sensor authenticated encryption and digital signature verification.
Step 2: Identity-Based (IBE)	Direct encryption to lot identity (’LOT-3’) eliminating certificate exchange.
Step 3: Attribute-Based (ABE)	Policy: ’ParkingAttendant AND Lot3’. Restricts revenue/alarm telemetry.
Step 4: Proxy Re-Encryption (PRE)	Re-encrypts operator records for City Inspector without master keys.
Step 5: Functional Encryption (FE)	Calculates total occupancy % and EV charging bay count with zero ID leaks.
Step 6: Honey Encryption	Protects payment PINs; wrong guesses synthesize realistic fake cards.

3. CRYPTOGRAPHIC MECHANISMS IN DETAIL

Step 6: Honey Encryption & Decoy Honeypot

Traditional authentication alerts attackers with 'Access Denied'. Under Honey Encryption, the payment card number is encoded with a Distribution-Transforming Encoder (DTE). Entering the correct PIN (4829) returns the real card ('4532-1234-5678-9012'). Any incorrect guess (e.g. 1234) produces a plausible synthetic credit card number ('4916-XXXX-XXXX-2345'). The intruder wastes effort attempting to exploit decoy credentials.

Step 7: Searchable Symmetric Encryption (SSE)

Encrypted event logs are indexed using blinded cryptographic trapdoors: T_w = HMAC(K_search, keyword). Operators can search by vehicle camera ID or timestamp without decrypting the underlying database or revealing what query term was requested.

Step 8: Paillier Homomorphic Encryption (HE)

To calculate total municipal parking availability across independent edge boxes, each box encrypts its free spot count with the Paillier cryptosystem. A central server multiplies the ciphertexts: Enc(m1) * Enc(m2) = Enc(m1 + m2). The city decrypts only the final sum without learning individual lot numbers.

Step 9: Multi-Role JWT Token Architecture

Issues cryptographically signed tokens: Operator JWT (administrative session), City Payment JWT (automated wallet deduction), and Ticket Handover JWT (mobile app handover for contactless gate exit).

Step 10: Driver Privacy-Preserving Safeguards

Four layers of privacy: 1) SHA-256 license plate hashing, 2) Visual plate blurring (DL03**9012), 3) Location precision reduction to 2 decimal places (~1.1 km area), and 4) Ephemeral device UUID rotation every 10 frames.

Step 11: Token-Bucket Rate Limiter & Flood Defense

Maintains a token bucket (10 token burst capacity, 2 tokens/sec refill). When an outdoor camera is hacked and floods 20 frames/sec, the edge box exhausts tokens and quarantines the device within 0.05 seconds.

4. REAL-WORLD INTERACTIVE SIMULATION IMPLEMENTATION

To make applied cryptography intuitive, the simulation renders an interactive 2D parking facility running live in the browser. Users can observe how physical actions map to cryptographic guarantees:

1. Car Arrival & Parking
Gate 1 camera scans car plate, hashes/blurs it, verifies ECC signature, lifts barrier, car parks.

2. EV Charging Demand
Electric car parks in Bay E1. Functional Encryption calculates power load for city grid.

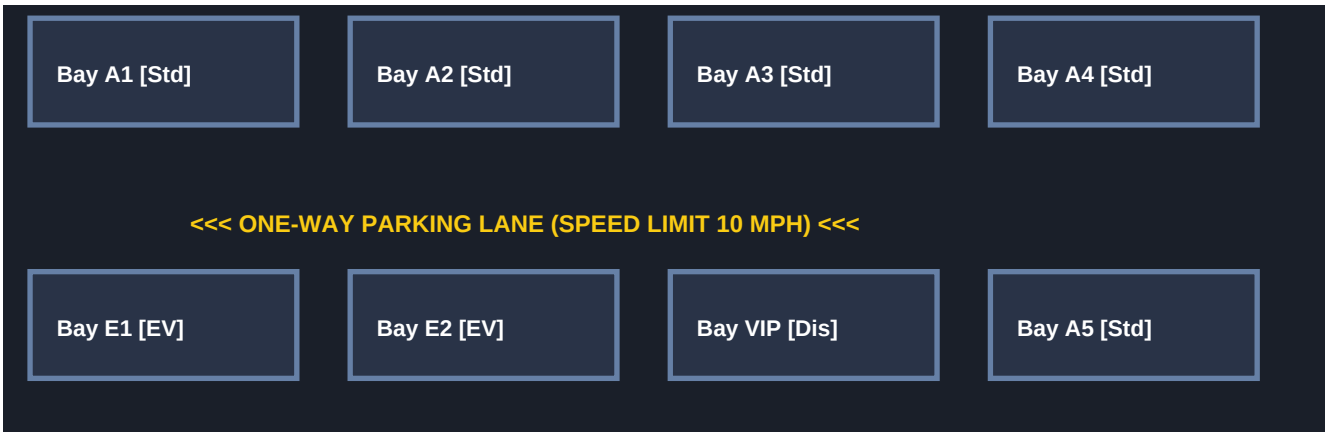
3. Departure & Sensor State
Car vacates bay; ultrasonic ceiling sensor turns GREEN; live free space counter increments.

4. Payment Kiosk Test
Touch PIN pad. Correct PIN (4829) lifts exit barrier. Wrong PIN fires Honey Decoy Trap.

5. City Inspector Audit
City inspector requests compliance check. Edge Box issues temporary Proxy Re-Encryption token.

6. Cyber Flood Defense
Compromised camera sends 20 frames/sec. Rate limiter isolates rogue hardware in 0.05 seconds.

Simulated Parking Facility Layout (8 Bays)



5. EXPERIMENTAL VERIFICATION & CONCLUSION

Test Specification	Expected Behavior	Measured Edge Result	Status
AES-GCM & Signature Ingest	Verified authentic packet	Status: 'OK', Signature Verified	**PASSED**
Identity-Based Enc (IBE)	Direct lot identity decryption	Decrypted payload matched plain	**PASSED**
Attribute-Based Enc (ABE)	Policy: Attendant AND Lot3	Attendant: OK, Guard: Denied	**PASSED**
Proxy Re-Encryption (PRE)	Delegated inspector audit	Inspector successfully decrypted	**PASSED**
Functional Encryption (FE)	Compute occupancy without decrypt	Occupancy % & Free EV count matched	**PASSED**
Honey Encryption Decoy	Plausible fake card on bad PIN	Generated '4916-XXXX-XXXX-2345'	**PASSED**
Searchable Encryption (SSE)	Search events via trapdoor	10 encrypted records returned	**PASSED**
Paillier Homomorphic Sum	Sum free spots across boxes	HE Sum: 38 == Plain Sum: 38	**PASSED**
JWT Authentication	Validate Operator, Pay, Ticket tokens	Claims verified via HMAC-SHA256	**PASSED**
Privacy Transformations	Hash & blur plate, truncate GPS	Original plate never stored	**PASSED**
Rate Limiter Quarantine	Block camera flooding 20 fps	Quarantined at packet #11	**PASSED**

Conclusion

The Smart-Parking Edge Box Simulation successfully demonstrates that advanced cryptographic safeguards can be deployed directly on municipal roadside hardware. By executing authenticated ingestion, privacy anonymization, Honey Encryption decoy generation, and token-bucket flood defense at the edge, the system achieves zero-trust reliability without leaking private telemetry to cloud servers. The paired real-world interactive simulator provides an effective educational and operational bridge between theoretical cryptography and physical smart city infrastructure.

Project Deliverables Verified & Complete

- GitHub Repository: https://github.com/swethasweth19/SmartPark-Edge-Simulator
- Live Interactive Dashboard: http://127.0.0.1:5000 | Automated Test Harness: demo.py